

Evaluating behavior trees

Simona Gugliermo^{a,b,*}, David Cáceres Domínguez^{a,1}, Marco Iannotta^{a,1}, Todor Stoyanov^a, Erik Schaffernicht^a

^a AASS Research Centre, Örebro University, Örebro, Sweden

^b Autonomous Transport Systems, Scania CV AB, Södertälje, Sweden

ARTICLE INFO

Keywords:

Behavior trees
Robotics
Artificial intelligence
Behavior-based systems

ABSTRACT

Behavior trees (BTs) are increasingly popular in the robotics community. Yet in the growing body of published work on this topic, there is a lack of consensus on what to measure and how to quantify BTs when reporting results. This is not only due to the lack of standardized measures, but due to the sometimes ambiguous use of definitions to describe BT properties. This work provides a comprehensive overview of BT properties the community is interested in, how they relate to each other, the metrics currently used to measure BTs, and whether the metrics appropriately quantify those properties of interest. Finally, we provide the practitioner with a set of metrics to measure, as well as insights into the properties that can be derived from those metrics.

By providing this holistic view of properties and their corresponding evaluation metrics, we hope to improve clarity when using BTs in robotics. This more systematic approach will make reported results more consistent and comparable when evaluating BTs.

1. Introduction

Behavior Trees (BTs) are task-switching structures used in the fields of Artificial Intelligence (AI) and robotics to control the behavior of agents or robots [1,2]. Although originally introduced in the gaming industry for the purpose of controlling non-player characters in games [3,4], BTs have recently found a growing number of applications in robotics [5,6] — ranging from low-level control [7], through high-level task switching [8,9], to multi-agent coordination [10,11]. BTs are popular in the robotics community, because they simplify the design of reusable and reactive behaviors that can be combined in a flexible and scalable way to achieve complex goals. The design of a BT focuses on actions instead of states, where each behavior is composed of a set of action nodes arranged in a hierarchical structure. This makes BTs suitable for creating modular behaviors that can easily be maintained and modified, while also providing a fault-tolerant framework for recovering from unexpected events or errors [1]. Thus, BTs provide a task-switching scheme seemingly ideal for enabling robots to adjust rapidly to unforeseen circumstances, recover from errors, and operate effectively and securely in dynamic and unpredictable environments.

While BTs have a lot of potential to serve as central robot behavior control structures, there is still a degree of ambiguity in the field when it comes to comparing different BT policies. When designing robot behaviors in general, it is common to assess performance on the basis of

some measurable properties — that is, based on well-established characteristics or attributes of a robot's behavior, used to describe, analyze and evaluate its ability to achieve a particular goal. For example, in a human-collaborative manufacturing environment, the behavior of the robot may need to prioritize both *safety* and *efficiency*. *Safety* ensures that the robot is not harming any human co-worker, while *efficiency* is necessary to minimize resource consumption (e.g., computational power, energy, materials). However, when robot control policies are encoded as BTs, the community has not yet reached a consensus on the methods to measure and compare the degree to which certain properties are satisfied. Even more troubling, there is currently a degree of ambiguity with regard to the *meaning or definition* of the desirable properties of BTs.

We believe this lack of standardization is a consequence of the adoption of BTs in different domains such as game industry and robotics, and the use of definitions borrowed from other fields such as software engineering. Therefore, property definitions are often open to the author's interpretation, which presents a significant barrier to the advancement of the field by hindering scientific discussions based on a common understanding of terms. Even in cases where a common definition is self-evident, devising metrics to measure the properties of robot behaviors can be a complex task that requires evaluating the system under different conditions and scenarios. As a consequence, it

* Corresponding author at: AASS Research Centre, Örebro University, Örebro, Sweden.

E-mail addresses: simona.gugliermo@oru.se, simona.gugliermo@scania.com (S. Gugliermo).

¹ Denotes equal contribution.

is frequent to read claims of one BT policy being “more reactive” or “more efficient” than another one [12,13], often based merely on visual interpretations or oversimplified metrics that do not directly measure the said property.

Current surveys of the field of Behavior Trees in robotics [2,14] and introductory overview texts [1] concentrate on formalizing, classifying and categorizing existing works, and on providing recommendations of best practices when applying BTs to specific problems. While these works serve an important function in the community, they do not sufficiently address the questions of measuring, evaluating and comparing BT policies. This deficiency in the community is a limiting factor, as different sub-communities, or sometimes even different research groups, might have distinct and even irreconcilable definitions for commonly used terms referring to BT properties, such as e.g., interpretability or efficiency. There is thus a marked need for a different kind of survey of the field — focusing instead on the properties of BTs in robotics and the measures through which the community commonly evaluates and compares BT policies. In this article, we meet that need and address the lack of common property definitions and comparison metrics, in an attempt to improve our current understanding of BTs as robot control policies. We identify and define the main properties used to evaluate BTs and discuss appropriate quantitative measures for each of these properties. In detail, based on a systematic literature analysis, we (i) collect and classify the metrics currently used for BTs, (ii) collect and formally define the main BT properties, and (iii) identify the relation between metrics and properties. This leads to critical considerations on what is currently missing in the state of the art and suggestions for future work.

2. Behavior trees

A BT is a directed acyclic graph composed of a root node and at least one internal node and one leaf node. The execution of a BT begins at the root node, which generates signals, called *ticks*, at a given frequency. These ticks propagate down to the leaf nodes through the internal nodes. Each node is executed only when it receives a tick and returns a status to its parent, which can be *Running*, *Success*, or *Failure*.

Internal nodes, referred to as Control Nodes, dictate the execution order of their child nodes. In detail, Sequence nodes execute their children in a depth-first order, returning *Success* only if all children succeed. This is analogous to the logical operator AND. Fallback nodes, also execute in a depth-first order, returning *Success* as soon as any child succeeds, and *Failure* only if all children fail. This is analogous to the logical operator OR. Parallel nodes, on the other hand, execute their children simultaneously, returning *Success* if M out of the N children return *Success*. This is analogous to the execution of the logical operators OR when $M = 1$ and AND when $M = N$. Lastly, a Decorator node modifies the return status of a single child node based on a custom policy.

Leaf nodes, or Execution Nodes, can be categorized into two types. An Action Node performs a specific command when ticked and returns *Success* if the command is successfully completed, *Failure* if it fails, and *Running* if it is still ongoing. On the other hand, a Condition Node checks a proposition and returns *Success* if the condition holds, and *Failure* otherwise.

3. Metrics for behavior trees

The goal of this section is to provide a comprehensive overview of the different metrics that can be used to measure BTs. These metrics comprise those sourced from the existing literature, as well as those newly formulated by the authors themselves, labeled as “NEW” in the manuscript. The metrics are further categorized into *Functional* and *Non-Functional*, with each category being thoroughly analyzed. In detail, Functional metrics assess whether a given BT satisfies the desired requirements and executes its intended functions. On the other hand,

Non-functional metrics measure aspects of the BT that are not directly related to its functionality, and apply regardless of its performance or behavior in a specific context.

Note that metrics can be defined to consider both functional and non-functional aspects simultaneously. For example, the Progress Synchronization Distance metric [15] takes into account both dimensions.

In our review, we explicitly exclude papers that assess the performance of BTs through the lens of user-defined functions. While these functions may be suitable to assess functional properties in certain scenarios, they heavily rely on the desired outcome of the specific task, making it difficult to formulate general considerations. Specifically, fitness functions commonly used to learn BT structure [12,16], aim to optimize diverse components like execution time and tree size. Nevertheless, such evaluations frequently often fail to target a singular property of BTs.

3.1. Functional metrics

Use of Resources

The metric aims to quantitatively evaluate the optimization of resource allocation, management, and utilization by a robot in the execution of its designated task.

One example of a resource is the *execution time*, which refers to the duration required for a BT to complete its execution [17,18], or it can represent the duration of a single node execution. The assessment of the execution time of individual nodes proves particularly valuable when evaluating the effects of introducing new specific nodes [15].

Predictability Distance

This metric was originally defined by Colledanchise [15,19] for evaluating parallel behaviors. The approach consists of defining a target progress value and then calculating the average deviation between the expected and the actual time at which the action reaches progress closest to the desired value. While primarily utilized in the context of parallel behaviors, it can potentially be used in sequential behavior scenarios as well.

Success Rate

Success rate can be defined in multiple ways, contingent upon the particular aspect we intend to measure. The most common usage pertains to the global or overall success of execution, which we define as the Desired Behavior Success Rate. When focusing, instead, on the local success of execution (i.e., single nodes or sub-trees), two metrics can be defined: the Single Node Success Rate and the Accuracy for Behavior Usage. These metrics prove particularly valuable in scenarios where the BT can achieve its intended goal, yet may encounter failures during execution. They also offer insights when we wish to focus on the success of a specific node or behavior within the broader system.

- **Desired Behavior Success Rate** evaluates the extent to which the behavior executed by the BT aligns with the behavior expected by a behavior oracle. It focuses solely on the desired behavior while disregarding the internal mechanisms of the BT itself (e.g., return status). The concept of desired behavior is adaptable to the specific property under evaluation. For instance, in assessing *safety*, the desired behavior may pertain to the avoidance of collisions with humans or obstacles. Conversely, when evaluating *correctness*, the desired behavior is defined by the correct execution of the designated task. For its flexibility and effectiveness, this metric stands out as the foremost and prevalent approach when referring to success rates [7,20]. In this metric, we also include cases where only a single evaluation is performed, intending to inspect the behavior in a single deterministic scenario that does not require multiple evaluations (e.g., [21–23]).

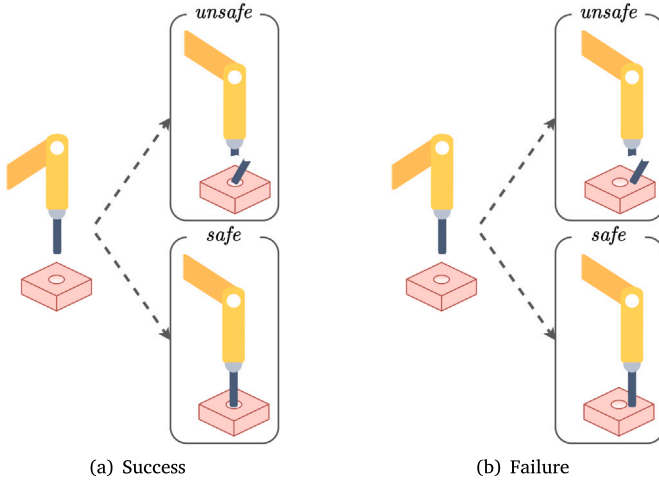


Fig. 1. Peg-in-hole scenario. Fig. 1(a) showcases a successful instance where the action node succeeds, increasing the Single Node Success Rate. In contrast, Fig. 1(b) depicts a scenario of an action node returning failure. In both scenarios, the operation can be deemed unsafe, increasing the Unsafe State Count, or safe, avoiding irreversible consequences.

- **Accuracy for behavior usage** [24] defined as the ratio between the correct executions of the inner sub-behaviors over the total expected executions. BTs typically consist of multiple modular components, making it pertinent to assess the successful completion of specific branches or behaviors within the tree. For instance, let us consider a BT with a root node that branches into three main sub-trees: the first one responsible for the battery charging behavior, the second one for moving to a conveyor, and the last one for loading the object. In some cases, our primary interest might lie in determining the success of the “charge battery” behavior. As a result, we focus on evaluating the performance of the sub-tree responsible for carrying out that specific behavior, regardless of whether the object is loaded or not.
- **Single Node Success Rate** refers to the ratio between the number of successful completions of a node $n \in N$ (i.e., when it returns *Success*) and the total number of executions of the node, including both successful and failed attempts, denoted by n_S and n_F respectively. Mathematically, it is expressed as:

$$SR_n = \frac{n_S}{n_S + n_F}$$

Unsafe State Count (NEW)

The goal of this metric is to assess and quantify the frequency with which the agent enters states that are considered unsafe. According to the definition of *safety* (Section 4, Definition 4.3), a state is unsafe when it leads to irreversible undesired behaviors within a system or environment.

By measuring this aspect of the agent’s behavior, we aim to gain valuable insights into its safety performance and identify potential issues that might arise due to unsafe state occurrences.

Given a set of unsafe states U , the USC metric is computed as the number of times the agent encounters states in U by the total number of states T visited during the execution:

$$USC = \frac{n_U}{n_T}$$

A higher USC value indicates that the agent frequently encounters hazardous states, thus leading to potential safety issues. Conversely, a lower USC value indicates a more robust and safer agent, as it encounters unsafe states less frequently.

It is important to note that the USC does not directly correlate with the success or failure rates of individual actions or nodes within the agent’s decision-making. Instead, it focuses solely on the identification and quantification of unsafe states encountered during execution, irrespective of the underlying mechanisms or outcomes of specific actions. USC measures the frequency of encountering states during BT execution that are deemed unsafe, irrespective of node success. Notably, a node can achieve success even within an unsafe system state, highlighting the necessity of USC as a complementary safety metric. To illustrate the distinction, let us consider a peg-in-hole scenario (Fig. 1), a common task in robotics where a peg-like object must be precisely inserted into a corresponding hole. The robot must apply an optimal amount of force to successfully insert the peg without causing damage to its end-effector or the surrounding components. In detail, in Fig. 1(a), the robot successfully inserts the peg into the hole, achieving success in the “insert peg” action. However, in one scenario applying adequate force ensures a safe insertion without encountering any unsafe states ($USC = 0$), while in the other case, excessive force leads to an unsafe success. Despite achieving the same Single Node Success Rate with successful peg insertion, the USC increases as we transition into an unsafe state. Conversely, Fig. 1(b) shows a scenario where the insertion fails because of misalignment between the peg and the hole. This failure can manifest in two distinct outcomes: a safe failure, where the peg remains intact despite not being inserted, and an unsafe failure, where the peg breaks, risking assembly integrity and increasing the USC .

Condition Check Frequency (NEW)

Condition Check Frequency (CCF) quantifies the frequency at which a specific Condition Node is evaluated (i.e., ticked) over a given time interval. It can be represented as:

$$CCF = \frac{n_{evaluations}}{\Delta T},$$

where $n_{evaluations}$ is the number of times a specific condition is checked and ΔT is the considered time interval. A higher CCF indicates that a certain Condition is checked more often, resulting in more timely updates of the BT decision-making process in response to environmental changes. This aspect makes this metric suitable for evaluating the BT reactivity, as discussed in the next section.

3.2. Non-functional metrics

Tree Dimensions

An analysis of the BT dimensions, which includes attributes such as height, width, and number of nodes, can be of critical importance in studying BT properties. Notably, as a non-functional metric, the scrutiny of tree dimensions is valuable during the BT design phases and the assessment of autonomously generated BTs.

- **Tree Height** is the total number of edges from the root node to the leaf node in the longest path [25].
- **Tree Width**, used in [26], refers to the maximum number of nodes that exist at any level within the tree structure.
- **Number of Nodes** refers to the count of individual elements (nodes) in a BT. Note that this metric does not distinguish between different types of nodes in a BT, such as control and execution nodes, and instead provides a holistic count of all nodes within the BT. This metric has gained significant prominence within the field, as indicated by the state-of-the-art research [8,27,28], for evaluating not only the structural characteristics but also various other properties of BTs, including their interpretability [27].

Graph Edit Distance

Graph Edit Distance (GED) is a measure of similarity between two graphs, which is defined as the minimum cost of transforming one graph into another by a sequence of edit operations. These can be

node insertions, deletions, or substitutions, as well as edge insertions, deletions, or substitutions [29]. Wathieu et al. [27] use GED to evaluate the degree of similarity between two BTs, while Iovino et al. [30] use it to evaluate the difference in modularity between a BT and a Finite State Machine (FSM) architecture.

Condition-to-Fallback ratio

This metric is defined as the ratio between the total number of condition nodes and the total number of fallback nodes in a BT. The CFR metric has been proposed by Gugliermo et al. [31] as an indicator of the efficiency of BT execution. Mathematically, it is expressed as:

$$CFR = \frac{\text{num_conditions}}{\text{num_fallbacks} + \epsilon},$$

where we have introduced the infinitesimal value ϵ to prevent division by zero errors. This adjustment ensures that scenarios without fallback nodes are appropriately considered.

Action Granularity (NEW)

Action granularity refers to the degree to which an action can be broken down into smaller and more specific sub-actions or components. The relative nature of Action Granularity allows for its application in a comparative manner, enabling the assessment of multiple BTs to determine which one exhibits a higher level of action decomposition. In a more granular BT, multiple action nodes can be combined to create a single action node in a less granular BT. For example, if two BTs encode the same behavior and have the same functionalities, the one with more action nodes is considered more granular. In a broader scenario where the BTs do not encode exactly the same behavior, we can compare actions by looking at their preconditions and effects. By combining two or more actions in the more granular BT, we check if the resulting action has the same preconditions and effects as a single action node in the less granular BT. However, in this paper, our primary focus is to compare Behavior Trees (BTs) with the same behavior specifically in terms of action granularity. As such, we use the number of action nodes as the sole metric to measure this aspect. For the more general case, causal analysis tools [32] are available to identify preconditions and effects, enabling a general comparison of action granularity.

4. Properties

In this section, we provide a comprehensive overview of the properties used in the BT community and the interrelation between them. We observe that, as done for the metrics, the properties can also be grouped into *Functional* and *Non-Functional*. On one hand, *Functional* properties typically refer to the ability to achieve specific goals that depend on the BT execution. On the other hand, *Non-Functional* properties are not related to the BT functionality and they can be assessed without the need for BT execution.

In detail, we first provide a definition for each of the properties that allow to evaluate the extent to which BTs can fulfill the intended functionalities and requirements. We base our definitions on the state of the art, clarifying the existing ambiguities in the literature and accompanying each property with an example. Then, we suggest design recommendations aimed at improving how the BT satisfies that particular property. Finally, we discuss how each property can be evaluated through the application of the metrics presented in Section 3, clarifying the conditions under which they can be applied. In particular, we distinguish between *General (G)* and *Application Specific (AS)* metrics. A *General* metric can always be used for the considered property, regardless of its application, while an *Application Specific* one is relevant only in some specific contexts. We encapsulate the evaluation considerations discussed in detail for each property in Table 1, providing a comprehensive overview of the connections between metrics and properties. Therefore, this table serves as a valuable reference, summarizing the key insights and providing general guidance on the metrics to consider when assessing particular properties.

4.1. Functional properties

Reactivity

We rely on the general definition of *reactivity* for control architectures [33]:

Definition 4.1 (Reactivity). A BT is reactive if its decision-making depends *only* on the current state of the environment, regardless of the execution of the correct action.

Colledanchise [1] refers to *reactivity* as the ability to quickly and efficiently react to changes (or perturbations). This entails that when there is a change in real-world conditions, the BT halts its current execution to react to the change. It is important to emphasize that *reactivity* solely pertains to the ability to react to the unexpected event, without being related to the subsequent action taken in response to it (which is related to *correctness* and *robustness*, see Definitions 4.4 and 4.5).

In general, *reactivity* is an intrinsic property of BTs as a result of repeated ticking, which frequently polls the environment for its current state. However, the design of the BT affects *reactivity*. For instance, nodes with memory prevent some nodes from ticking, thus resulting in a less reactive BT. To illustrate how BTs that encode the same behavior can perform differently in terms of *reactivity*, we refer to Fig. 2. The figure shows two different BTs designed for the task of moving a robot to a specific goal location, while monitoring its battery level and charging the battery if under a certain threshold. The two BTs rely on the same Condition and Action Nodes. However, they exploit different Control Nodes and, consequently, differ in the arrangement of the Condition Nodes (which are mutually exclusive). Although both BTs encode the same behavior, the BT in Fig. 2(b) results in a more reactive behavior, due to the use of the Parallel and the Sequence Nodes. In this particular BT, the Parallel Node simultaneously ticks the right part of the tree, while performing the *Move to goal* action. When the battery level is less than the specified threshold, the *Battery <10%* Condition on the right side returns *Success* and the right sequence immediately propagates the tick to the *Charge* Action, which will be performed. On the contrary, in the BT of Fig. 2(a), during the execution of the *Move to goal* action, no more ticks are propagated on the right side of the tree. In this case, assuming that the Sequence and the left Fallback node are reactive, when the battery level is less than the specified threshold, the *Battery <10%* Condition on the left side returns *Success*. The tick is then propagated to the right side of the tree, where the *Battery ≥ 10%* condition is ticked before the *Charge* action is finally executed. In conclusion, the difference in *reactivity* between the two considered BTs derives from the number of nodes that have to be ticked before performing the correct action w.r.t. the new environment state. Fewer nodes to tick in sequence results in less time to start executing the new action and, consequently, in a more reactive behavior.

Design recommendations. When designing BTs to prioritize *reactivity* to certain conditions, several key principles should be considered. When possible, it is important to position the parts of the tree that need to react promptly closer to the root and towards the left side of the tree, reducing the distance the tick needs to travel before returning a status. Moreover, minimizing the overall number of nodes, resulting in a more compact BT can also increase the *reactivity* of the BT. Additionally, introducing parallel nodes can significantly influence *reactivity* by enabling the concurrent execution of multiple behaviors.

Evaluation. As delineated in Table 1, the execution time can be considered as a *General* metric for assessing *reactivity*. This entails measuring the duration required to respond to a perturbation and start the execution of the subsequent action. For instance, Neufeld et al. [34] measure the time required to achieve the goal in the planning context to evaluate that BTs are more responsive than HTN planners.

Additionally, one more *General* metric is the newly introduced Condition Check Frequency. Monitoring the tick frequency of specific Condition Nodes provides valuable insights into BT *reactivity*, as a higher frequency indicates that the BT decision-making relies on a more recent status of the environment.

The *Application Specific* metrics highlighted in Table 1 for assessing reactivity derive from the observation that, as explained later, *robustness* can be seen as a combination of *correctness* and *reactivity* when considering environmental changes at run-time (see Definition 4.5 and further discussion). For this reason, in cases where the response time is not the primary focus, metrics such as Behavior Usage Accuracy and Desired Behavior Success Rate (typically used to assess *robustness*) can be employed as well as *Application Specific* metrics. These metrics would not only capture the *reactivity* to specific perturbations but also the ability to perform the correct action. It is, therefore, possible to assess whether the BT is exhibiting the reactive and correct behavior, regardless of the reaction time. For example, Cáceres Domínguez et al. [7] evaluate the *reactivity* of a robot manipulator in a pick, place, and push scenario, by considering the Desired Behavior Success Rate when a global disturbance is artificially introduced during task execution. In this case, the desired behavior is task completion in case of an applied disturbance. While this does not give any information about the reaction time of the system, it provides valuable insight into the system's capability to react to the updated status of the environment. The drawback of the use of these metrics is that they do not take into account the scenarios where the BT has reacted to the perturbation but has executed an incorrect action. In the same way, the Single Node Success Rate can also serve as an *Application Specific* metric, especially when there is a designated "critical" Condition Node that requires a specific reaction. Therefore, it allows for the evaluation of the BT *reactivity* in responding appropriately to the identified critical condition.

Efficiency

When referring to *efficiency*, the BT community often focuses on *time efficiency* [35]. However, we prefer to adopt a more general definition that considers a wider range of resources, as typically intended in the context of software engineering [36].

Definition 4.2 (Efficiency). The degree to which a Behavior Tree performs its designated behavior with minimum consumption of resources.

To illustrate how BTs that encode the same behavior can perform differently in terms of *efficiency*, we refer again to Fig. 2. Earlier, we discussed the concept of *reactivity* and highlighted that the behavior modeled by the BT in Fig. 2(b) is more reactive than the one in Fig. 2(a). On the other hand, in terms of *efficiency*, the BT shown in Fig. 2(a) exhibits a more efficient behavior. This follows from the observation that ticking both parts of the tree (the left and the right one) at the same time in the BT of Fig. 2(b) introduces a redundant check of the conditions that requires a higher degree of resource consumption (despite increasing the *reactivity* of the resulting behavior).

Design recommendations. When *efficiency* is crucial in the application, the design should aim to minimize the consumption of resources. This can be achieved by avoiding unnecessary condition checks, which can be computationally costly. Additionally, the BT should be designed in a way that strictly necessary conditions are checked, and unnecessary ones are avoided, saving the use of resources. Lastly, while parallel nodes can enhance *reactivity*, they should be used sparingly in the context of *efficiency*. Excessive use of parallel nodes can lead to resource contention and reduce the overall *efficiency* of the BT.

Evaluation. As shown in Table 1, *efficiency* assessment commonly relies on the Use of Resources as a primary *General* evaluation metric. It should be noted that the choice of a resource for evaluation may vary depending on the context and requirements. Therefore claims on *efficiency* are dependent on the evaluated resource. Various applications,

from swarm systems [37] to robot manipulation [9,17], often employ time in seconds as a measure of *efficiency*.

In Table 1, we also list alternative metrics that can be utilized to evaluate *efficiency* in specific contexts.

Predictability distance can serve as an *Application Specific* measure of *efficiency*, as a higher predictability distance often results in longer execution times. Therefore, in certain scenarios, this metric can be utilized to compare the *efficiency* of different BTs.

Furthermore, non-functional metrics have gained attention as a means of assessing *efficiency*, ranging from the size of the tree [13] to Condition-to-Fallback ratio [31]. An illustrative instance of the Condition-to-Fallback ratio's applicability can be observed in Fig. 2. Specifically, Fig. 2(a) presents a smaller *CFR* ($CFR = 1$) in contrast to Fig. 2(b) ($CFR \rightarrow \infty$), thus resulting in a more efficient BT. However, it is essential to emphasize that non-functional metrics are solely based on the BT's design and should not be regarded as a definitive way to evaluate its overall performance. Instead, it serves as a useful hint during the BT design process to create a more efficient behavior tree in some applications.

Safety

According to the definition in [1], *safety* in Behavior Trees refers to:

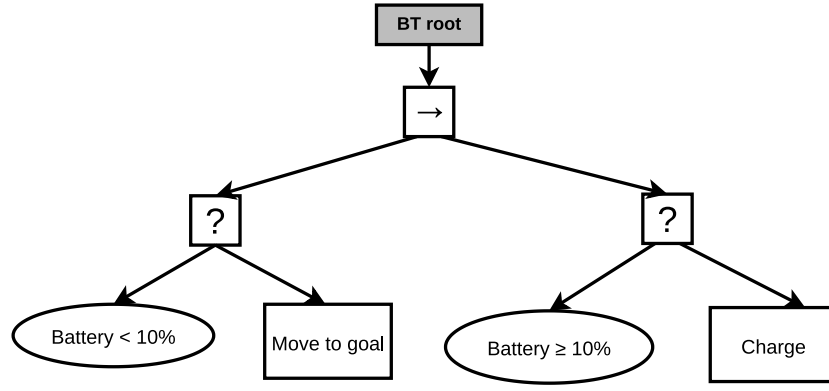
Definition 4.3 (Safety). The ability to avoid specific parts of the state space that can result in irreversible undesired behaviors.

Safety in BTs is crucial for ensuring that autonomous systems can operate consistently while avoiding catastrophic outcomes. In detail, a BT is considered safe if:

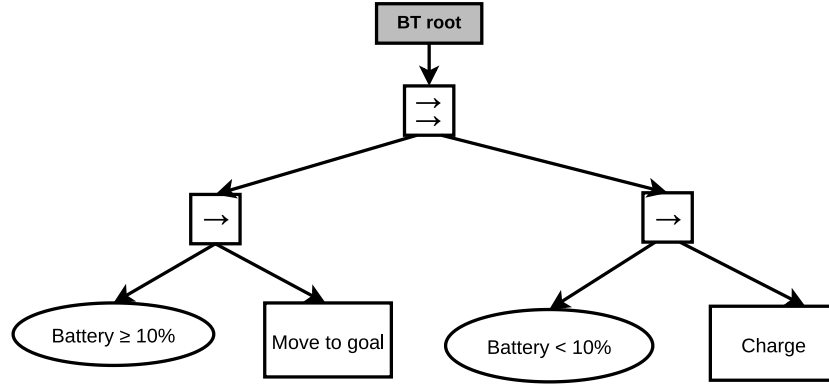
1. All action nodes are safe. That is, if the state $x(t)$ is safe and we execute action a , we are guaranteed to transition to $x(t+1)$ that is safe. Actions can be considered safe, given a set of pre-conditions $c(x(t))$ hold.
2. The BT design guarantees that all actions a_i are always executed only in states that satisfy their pre-conditions.

The specification of relevant conditions for each action plays a crucial role in reinforcing the above considerations. For instance, Fig. 3 illustrates an example of two BTs aimed at achieving the goal of picking up an object. Although both BTs have the same goal, they exhibit different behaviors. Fig. 3(b) incorporates safety considerations, such as the robot's capabilities in terms of weight and size. This ensures that when performing the action *pick*, the transition from $x(t)$ to $x(t+1)$ occurs in a safe manner. On the other hand, Fig. 3(a) solely focuses on executing the specified actions without verifying the safety conditions, leading to a potentially unsafe state $x(t+1)$.

Design recommendations. Guaranteeing *safety* in the design of BTs is a complex and challenging task. This is because the *safety* of a BT is not just dependent on its design, but also on the behavior of other systems it might interact with and the dynamics of the environment. Moreover, the interactions between different parts of the BT can lead to emergent behaviors that are difficult to predict and control, hence to evaluate *safety*, it is required to test the BT in real-world scenarios. Despite these challenges, several key principles can guide the design process to enhance *safety*. *Safety* conditions and actions should be prioritized by placing them close to the root of the tree, ensuring their prompt execution and increasing the *reactivity* of the BT to safety conditions. Conditions should be checked explicitly, ensuring that critical actions are executed only in states that satisfy their safety conditions. Generally, the *reactivity* of safety conditions and actions should be increased (see 4.1), allowing the BT to quickly react to changes and enhance its *safety*. While these principles can guide the design of safer BTs, it is important to remember that *safety* cannot be guaranteed and these measures should be complemented with rigorous testing and validation.



(a) Less reactive. More efficient.



(b) More reactive. Less efficient

Fig. 2. Reactivity and efficiency examples. Both BTs encode the same behavior using different control nodes. Note that condition nodes represent mutually exclusive conditions.

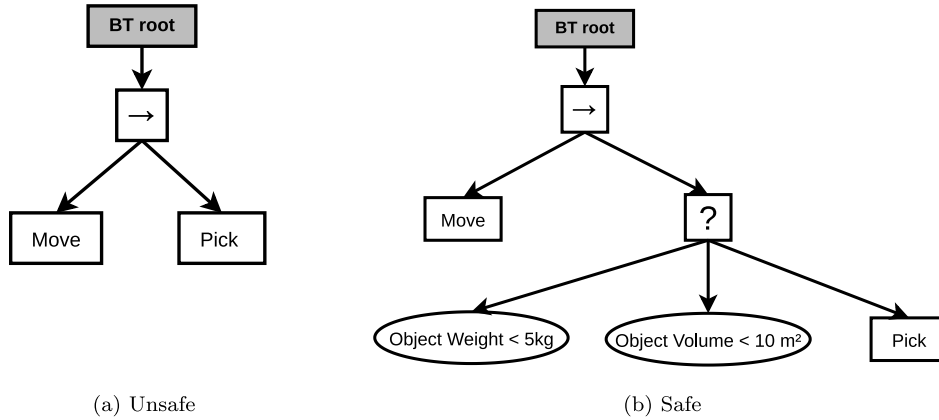


Fig. 3. Safety example. The two BTs have different structures and different behaviors.

Evaluation. To evaluate *safety*, we mainly consider the introduced Unsafe State Count and the Desired Behavior Success Rate, both classified as *General* in Table 1. In the case of the Desired Behavior Success Rate, the BT must be evaluated in scenarios that can potentially bring the BT to unsafe states (e.g., collision with humans). For example, Colledanchise et al. [23] test the robot behavior in critical scenarios (i.e., low battery level) by artificially injecting faults in the system. It is important to note that these metrics cannot guarantee *safety*. However, they provide valuable insights into the level of *safety* a BT exhibits under specific evaluation circumstances.

Moreover, as outlined in Table 1, to assess *safety* it is possible to use Single Node Success Rate and Accuracy for Behavior Usage under

Application Specific circumstances. The former is applicable when the BT incorporates condition nodes to model specific safety constraints. In this case, the Single Node Success Rate evaluates the BT's ability to adhere to the defined safety constraints. Likewise, the Accuracy for Behavior Usage metric can provide valuable insights into *safety* when the sub-behaviors within the BT are expected to exhibit correct or safe behavior.

Correctness and Robustness

In computer science, a program is considered correct if it is “free from faults in its specification, design, and implementation” [36]. In line with [38], we adopt the following definition:

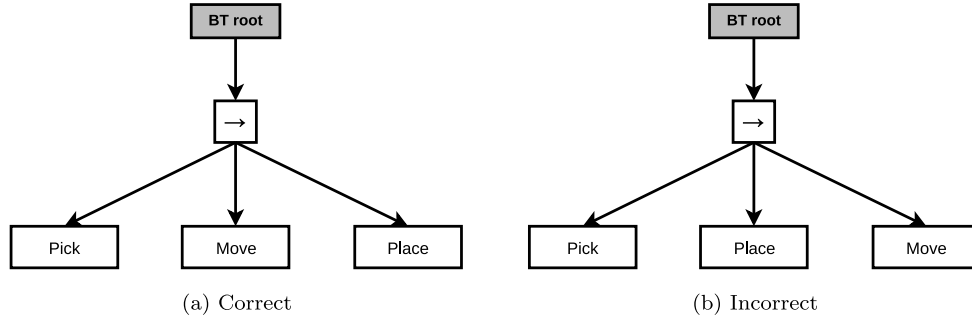


Fig. 4. Correctness example. Both BTs aim to perform a pick-and-place operation.

Definition 4.4 (Correctness). The *correctness* of a BT refers to its ability to exhibit the intended behavior throughout the whole execution.

A BT is considered correct if its execution reflects the intended behavior, not only in terms of the final results but also in terms of its intermediate steps. Specifically, the BT must adhere to the order and timing of the intended actions, as well as any constraints or requirements imposed by the task at hand.

To illustrate this, we can refer to Fig. 4 where the designed BTs aim to perform the tasks of picking, moving, and placing an object.

Although both BTs encode the same actions under the same conditions, they sequence the actions differently. On the one hand, Fig. 4(a) encodes the behavior of *pick*, *move* to the desired position, and then *place* the object. On the other hand, Fig. 4(b) encodes the behavior of *pick*, *place*, and then *move* to a desired position. Even though these two BTs might perform the tasks without complications,² only Fig. 4(a) can be said to be correct as it adheres its entire behavior to the one initially intended. Moreover, Fig. 2 serves as an example to acknowledge that there might be more than one BT design that can encode the same behavior. In this case, both BTs are equally correct as they result in the same intended behavior.

In dynamic environments, it can be crucial to assess *correctness* under potential changes. For this reason, we define *robustness* in the following way.

Definition 4.5 (Robustness). A BT is *robust* if it is correct in the presence of environmental changes.

The environmental changes mentioned in the definition above can be categorized into two types:

- *Large domain inputs.* These pertain to variations in the task domain, such as different positions or sizes of the object to grasp for a standard pick-and-place scenario.
- *Changes during BT execution.* This refers to modifications that may occur while the BT is already in progress.

Consequently, *robustness* refers to the ability to exhibit the correct behavior in the presence of at least one of the two aforementioned scenarios. It is important to underline that the second type of perturbation refers to the same changes at run-time taken into account when discussing *reactivity* (Definition 4.1). However, when referring to *robustness*, the focus is not only on the ability to react to the change in the environment but also to execute the correct action. This is why, for this specific scenario, *robustness* can be seen as a combination of *correctness* and *reactivity*.

Typically, when considering any BT, the more potential situations of failure it considers, the more robust it will be. As an example, the BT in Fig. 5 is considered to be more robust than uniquely considering the behavior encapsulated in the node labeled *Sub-tree*.

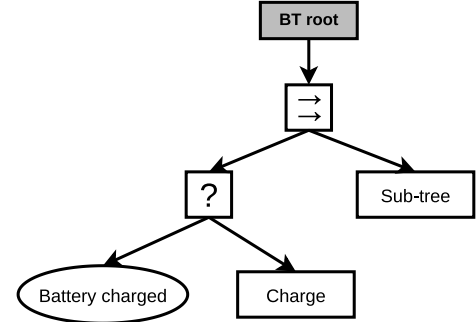


Fig. 5. Robustness example. Sub-tree can refer to any relevant operation such as the one encapsulated in Fig. 4(a).

Design recommendations. Ensuring the *correctness* of a BT is a complex task that requires careful analysis of the behavior during execution. There are no specific design recommendations for *correctness* as it is inherently tied to the specific behavior the BT is intended to exhibit. From the design point of view, it is crucial to ensure that the BT adheres to the order and timing of the intended actions, as well as any constraints or requirements imposed by the task at hand.

For *robustness*, the design of the BT should aim to increase its *reactivity*. This is because a robust BT is one that remains correct in the presence of environmental changes, and increased *reactivity* allows the BT to quickly adapt to such changes.

Evaluation. Table 1 shows that both *correctness* and *robustness* can be generally evaluated by measuring the Success Rate of the Desired Behavior and the Accuracy for Behavior Usage. When evaluating *correctness* in deterministic scenarios, a single evaluation may also be sufficient to inspect the BT behavior [21,22]. Regarding *robustness*, on the one hand, for evaluating it under large domain inputs, different task variations are typically considered [39]. On the other hand, artificially introducing perturbations during the task execution is a straightforward way to evaluate if the BT exhibits the correct behavior in response to the disturbance [7,40].

GED has also been used to evaluate *correctness* [27]. However, it is important to note that [31] highlights concerns about using this metric, as it neglects the semantics of the tree. Considering that semantics play a crucial role in evaluating both *correctness* and *robustness*, we do not recommend relying on GED as an evaluation metric.

4.2. Non functional properties

Modularity

In line with [36] we adopt the following definition.

² Under the assumption that the placing position is reachable by the robotic arm without performing the *move* action.

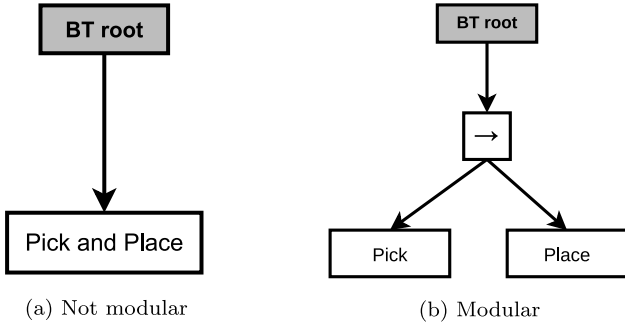


Fig. 6. Modularity example. The two BTs have different structures but encode the same behavior.

Definition 4.6 (Modularity). The degree to which a Behavior Tree is composed of decoupled sub-trees such that a change to one sub-tree has minimal impact on the others.

This definition aligns with the one presented in [1], as the property of a BT with independent sub-trees allows for the easy separation and recombination of building blocks. *Modularity* is a property that intrinsically characterizes BTs, where each sub-tree is an independent module, thanks to the common standard interface given by the return statuses.

The *modularity* of a BT is intricately linked to the level of granularity displayed by the actions it represents. Granularity, in this context, refers to the degree of detail and specificity in defining or decomposing actions. A higher level of granularity enables greater flexibility and reusability as individual sub-actions can be combined and rearranged to create various behaviors. Conversely, a lower level of granularity curtails *modularity* and limits system flexibility, as actions become tightly coupled and challenging to separate or rearrange.

To illustrate this concept, Fig. 6 presents a visual comparison of two BTs encoding a Pick and Place task. In Fig. 6(b), the action is partitioned into two distinct actions, resulting in heightened *modularity*. In contrast, the other case (Fig. 6(a)) demonstrates a lower degree of granularity, leading to diminished *modularity*.

Design recommendations. When considering *modularity* in design, it is crucial to enhance action granularity by separating actions into distinct behaviors rather than encoding multiple behaviors within one action. A key design suggestion involves analyzing conditions, potentially utilizing causal analysis tools [32], and identifying which conditions affect each action. This process helps create subtrees that contain only necessary conditions. For instance, in a pick-and-place scenario, an ideal approach would be to create two subtrees: one responsible for the picking behavior and another for the placing one. The picking subtree should exclusively incorporate conditions pertinent to picking, ensuring simplicity, clarity in behavior, and facilitating easy reuse. Similarly, the placing subtree should adhere to the same principles.

Evaluation. To assess *modularity*, we employ the introduced *Action Granularity* as a *General* metric, as shown in Table 1. When comparing two BTs, the BT with a higher Action Granularity denotes a greater level of action decomposition and, consequently, a higher degree of *modularity*.

In certain applications, comparing the dimensions of two BTs can offer insights into their *modularity*. For example, a higher number of nodes may indicate that a BT is more modular, with its behaviors and actions organized into smaller, reusable components. Conversely, a lower node count may suggest a more monolithic structure. While this is not a definitive measure of *modularity*, it can provide an indication of how the BTs are designed and organized in terms of their complexity and reusability.

Additionally, the use of GED can also contribute to understanding *modularity* in certain contexts [30]. For example, consider the scenario depicted in Fig. 6. If we had to introduce a new action, following the “pick and place” action, the comparison of the BTs in Figs. 6(a) and 6(b) illustrates the concept. In Fig. 6(a), the addition requires the inclusion of a sequence node connecting “pick and place” and the new node. In contrast, Fig. 6(b) demonstrates a more modular approach where merely adding a new action node, and linking it with the existing sequence node leads to a lower GED. Therefore, in this instance, GED can be used as a metric for evaluating *modularity*.

Interpretability (Readability-Transparency)

The concept of *interpretability* of a BT has been used for the first time in [27] based on Post Hoc Interpretation [41]. However, to the best of our knowledge, no formal definition of *interpretability* of a BT has been provided. Therefore, we define *interpretability* as follows.

Definition 4.7 (Interpretability). The degree to which the design of the BT conveys its expected behavior.

Improving the *interpretability* of a BT allows other users to better understand the behaviors and predict the agent’s response to any input without prior knowledge. We note that in prior works [42–45], the terms *transparency* and *readability* are also used for referring to the same concept.

Let us consider an example of two BTs and examine their *interpretability*. Both BTs in Figs. 6(a) and 4(a) encode the same behavior of a pick-and-place operation. However, they differ significantly in terms of *interpretability*. Fig. 6(a) encapsulates the entire behavior within one action node, offering limited insights into its internal workings. On the other hand, Fig. 4(a) exhibits a higher level of granularity, providing more detailed information about the behavior. This improved representation not only reveals a middle step of the *Move* action in the pick-and-place operation, but also presents conditions that offer valuable insights into when each action is executed, resulting in a more interpretable BT.

Design recommendations. To ensure that BTs are interpretable, it is important to clearly define conditions and actions during the design phase. This practice not only enhances *interpretability* within the context of BT design but also aligns with best practices in computer science. Despite the importance of clear definitions, achieving *interpretability* in BT design remains challenging. While some studies [27] suggest that a more compact BT improves *interpretability*, formal evidence supporting this assertion is lacking. This gap in research underscores the need for future exploration and investigation into what factors contribute to the *interpretability* of BTs.

Evaluation. Measuring *interpretability* is a common challenge in many fields. Siu et al. [46] studied how people understand formal specifications through a comprehensive user study. In the context of BTs, Pattaraporn T. et al. [47] evaluated a BT-based robot behavior creation system for non-expert users. Similarly to these works, we believe that meaningful comparisons of the BT Interpretability should be conducted through user studies, as indicated in Table 1. Collecting user opinions can help derive optimal practices for enhancing this property during the design phase. Although, under certain applications, measuring the Action Granularity, counting the number of nodes or evaluating the height and width of the tree might give some hints on the *interpretability* of the BT, this is always subject to the description encapsulated in the nodes. As a result, it necessitates measuring factors from the perspective of the user or designer, rather than relying solely on characteristics inherent to the BT itself.

Table 1

Mapping Metrics and Properties. Metrics and properties are classified as functional (F) or non-functional (NF), following the definitions provided in Section 3 for metrics and Section 4 for properties, respectively. Each metric-property pair is classified as General (Green) if the metric serves as a foundational benchmark for assessing that property across diverse contexts. Application-specific (Blue) metric-property pairs are tailored to a particular application scenario. Selecting and applying them requires careful consideration of the specific requirements and details of the property being evaluated. Non-Recommended (Gray) metric-property pairs indicate potential limitations or drawbacks that practitioners should carefully consider, if they want to apply the particular metric to the task of measuring the property in question. (See [50–72]).

	F					NF	
	Reactivity	Efficiency	Safety	Correctness	Robustness	Modularity	Interpretability
Use of Resources	* [34]	[9,17,37,50–65,72]					
Predictability Distance							
Accuracy for Behavior Usage				[24]			
Single Node Success Rate							
Desired Behavior Success Rate	[7,34]		[23,66]	[21,22,52,66–69]	[7,20,34,39,40,48,52–54,65,70,71]		
Unsafe State Count							
Condition Check Frequency							
Tree Dimensions		[13,57]					[26–28]
GED				[27]		[30]	
Condition-to-Fallback Ratio		[31]					
Action Granularity							
User Study							

* refers to time.

4.3. Interrelationship among properties

Each property plays a significant role in shaping the behavior of the robot and its performance. However, optimizing one property can potentially compromise another. By gaining a better understanding of how modifications in one property can influence the others, designers can make well-informed decisions that align with the specific requirements and objectives of a given system.

Properties can often involve trade-offs that need to be carefully considered during the design phase. One instance of this is the *efficiency-reactivity trade-off*, which has already been discussed in Section 4.1 and exemplified in Fig. 2. Another example is the *efficiency-correctness trade-off*, which we refer to as **effectiveness**. Notably, Effectiveness stands as a prominent concept within literature [13], albeit lacking a precise definition. Therefore, we define it as the degree to which a BT is correct and efficient at the same time. In practical scenarios, optimizing the execution of a BT by skipping certain evaluations may lead to erroneous outcomes or failures to fulfill the intended behaviors. On the other hand, prioritizing *correctness* without considering *efficiency* can result in unnecessarily complex or time-consuming tree evaluations.

A similar trade-off occurs when considering the balance between the number of nodes and *correctness*, giving rise to the **redundancy** [48, 49] property. *Redundancy* denotes the degree to which each sub-tree within the BT is indispensable for achieving the intended behavior. By evaluating the degree of *redundancy*, it is possible to determine whether all sub-trees are necessary for the desired outcome, or if there are redundant elements that can be eliminated without compromising the BT *correctness*. Moreover, *redundancy* can also affect *interpretability*. The presence of redundant nodes that lack logical contribution to the control policy increases the BT complexity, negatively impacting its *interpretability*. Thus it is important to consider the interplay between properties and synchronize with the particular needs and goals of the desired system.

4.4. Examples of evaluating BTs in real-world scenarios

Our aim in this section is to highlight how the discussion in Sections 3–4 relates to real-world applications of Behavior Trees. We illustrate how Table 1, which outlines how metrics and properties are related, can be useful in practical applications. Through concrete examples, our intent is to underscore the practical significance of our research and showcase how it can be effectively applied in addressing real-world challenges in the deployment of Behavior Trees for robotics problems.

The applications of BTs to robotics are numerous and diverse [2]. The properties of interest when considering BTs as policy representations are often predicated on the application — in different deployment

contexts, such as factories, warehouses, or search and rescue operations, different types of robots are used to perform specific tasks with specific KPI. For example, consider a robot assembly line used to manufacture electronic devices, in the absence of human co-workers. In this setting, *correctness* is an essential requirement to ensure that each step of the assembly process is properly executed in order to meet quality standards and functional specifications. To assess *correctness*, Table 1 suggests that it is generally possible to exploit the *Desired Behavior Success Rate*. For this reason, in our specific example, this metric can be used to ensure precise component placement and soldering processes. As for *safety* concerns, in this context, they include the integrity of the robotic systems and the production environment. Without human intervention, safety measures must focus on preventing collisions among robotic arms, conveyor systems, and other machinery, but crucially would not need to maintain stringent health and safety standards to protect human co-workers. Similar to the evaluation of *correctness*, the *Desired Behavior Success Rate* can also be used as a metric for evaluating *safety*, as indicated in Table 1. In this case, the behaviors being assessed may encompass potential misalignments of the robotic arm, which may pose risks to vital components, compromising the product quality and disrupting the production flow.

Another crucial property in this context is *efficiency*, primarily due to the application-specific demand of energy- and resource-efficient execution of repetitive tasks. Therefore, as highlighted in Table 1, the BT design phase requires careful consideration of factors like *tree dimensions*. Moreover, during the execution, minimizing the *use of resources* is crucial for improving *efficiency*. This can entail reductions in cycle times and energy consumption.

Another example within industrial factory settings involves considering a scenario where mobile manipulators are deployed to assist in material handling tasks alongside human workers. Imagine a large manufacturing facility where goods need to be transported between different stations. Mobile manipulators equipped with robotic arms are programmed to pick up items from one location and deliver them to another, while human workers are also present in the same workspace. In this context, *correctness* remains fundamental. The *Desired Behavior Success Rate* can still be used to measure how often the manipulator successfully completes the task without any errors. *Safety* considerations shift focus towards human–robot interaction, emphasizing the need to prevent accidents or injuries in dynamic environments where humans and mobile manipulators coexist. Human–robot interaction becomes crucial in this environment to prevent accidents or injuries. For instance, when a human worker operates close to a mobile manipulator that moves between locations, it is important to ensure that the manipulator slows down or stops to avoid collisions. This is the intended behavior we aim to achieve, and therefore the one that should

be evaluated with the *Desired Behavior Success Rate*. Additionally, as shown in Table 1, one could assess the *Condition Check Frequency* for this same setting, guaranteeing continuous safety checks to maintain the responsiveness of the sub-tree or nodes responsible for ensuring safety. By increasing these metrics, it is possible to improve safety in such settings. Finally, *reactivity* is important in situations where the environment is dynamic and unpredictable. In our factory setting, the ability of mobile manipulators to perceive changes in their surroundings and react is essential. For instance, if an unexpected obstacle appears in the manipulator's path, it should be able to quickly analyze the situation and choose an alternative route to reach its destination without delay. By parallelizing behaviors within the BT design, the *Condition Check Frequency* during execution is increased. This approach ensures that the manipulator can prioritize and execute tasks efficiently based on real-time information.

Unlike in factory settings, where trade-offs may be more straightforward, there are domains where all properties are equally indispensable. This is the case in the realm of autonomous vehicles which presents multiple challenges. In this context, *correctness*, *safety*, *reactivity*, *efficiency*, and *robustness* are all critical properties that must be carefully balanced. Indeed, *correctness* ensures that the vehicle accurately perceives its environment and makes correct decisions to navigate safely. *Robustness* is important to guarantee that the vehicle maintains its functionality in the face of unexpected or adverse conditions. *Safety* is paramount to prevent accidents or collisions with other vehicles, pedestrians, or obstacles. *Reactivity* is essential for autonomous vehicles to respond quickly to changing road conditions, unexpected obstacles, or sudden hazards. *Efficiency*, on the other hand, pertains to the optimal allocation and utilization of resources, including time, energy, and computational power. Therefore, in domains where all properties are equally indispensable, ongoing research and development efforts are essential to refine BT design and execution methodologies to meet the challenges of different real-world applications. While our research provides valuable insights into the expected trade-offs between different properties, it does not, nor can it, synthesize a single BT fitness function that would be suitable in all conceivable application scenarios. Combining the measures of different metrics into a single optimization objective is non-trivial and invariably necessitates choosing weighting factors, which may have unintended consequences. We identify this task as an important avenue for future investigation within the BT community.

5. Discussion

This paper aims to provide clarity within the robotics community regarding the properties of a BT, how they relate to each other, and the corresponding measurement metrics. The outcome of this study is presented in Table 1, which outlines the relationship between metrics and properties.

A straightforward analysis of Table 1 reveals a consistent pattern where each property aligns with at least one corresponding *General* metric. The application of *General* metrics in property measurement can be approached in a reliable and standardized manner. These metrics serve as fundamental benchmarks, enabling practitioners to assess BT behavior without necessitating specific justifications for their inclusion.

While *General* metrics provide a foundational assessment, the use of *Application Specific* metrics requires a more deliberate rationale. Practitioners are encouraged to find a solid motivation for the use of these metrics, considering the unique characteristics and goals of the application scenario. In certain instances, the use of *General* metrics might introduce higher resource costs compared to *Application Specific* alternatives, which may also provide different insights into the BT behavior. Therefore, practitioners may find it beneficial or even necessary to opt for *Application Specific* metrics, particularly when aiming for resource efficiency when evaluating non-functional properties.

Upon further examination of Table 1, it becomes evident that the success rate is the most prevalent among the functional metrics, which is a logical outcome of our focus on robotic behaviors. It can also be observed that currently functional metrics prove to be the most suitable approach for evaluating and effectively measuring functional properties. When it comes to non-functional properties, user studies currently stand as the most suitable method. Since conducting a user study is a challenging and time-consuming process, non-functional metrics are commonly employed as an alternative given their ease of use.

To summarize, we present a comprehensive overview of BT properties, the appropriate metrics for measuring them, and the ways in which these properties influence and correlate with each other. We provide practitioners with a recommended set of metrics to employ for measurement and to draw meaningful conclusions about the properties being evaluated.

For future work, it would be beneficial to establish non-functional metrics tailored to assess functional properties. This approach offers the advantage of assessing BT properties without having to execute the BT, thus avoiding some of the potential safety issues connected to real-world testing. In an attempt to address this, some researchers have experimented with using non-functional metrics to assess efficiency-related functional properties [31]. Nevertheless, this remains an open area of investigation, laden with many challenges that have yet to be addressed. It is important to note that exploring these applications comprehensively falls under future work and is outside the scope of this paper. Our primary objective here is to present the framework within the context of robotics.

To conclude, we stress the importance for BT users to exercise caution when making claims about BTs and to always consider potential trade-offs between the properties of interest. Being mindful of these factors will lead to a more informed and accurate analysis of BT behavior and its associated properties.

CRedit authorship contribution statement

Simona Gugliermo: Writing – review & editing, Writing – original draft, Visualization, Validation, Resources, Methodology, Investigation, Formal analysis, Conceptualization. **David Cáceres Domínguez:** Writing – review & editing, Writing – original draft, Visualization, Validation, Resources, Methodology, Investigation, Formal analysis, Conceptualization. **Marco Iannotta:** Writing – review & editing, Writing – original draft, Visualization, Validation, Resources, Methodology, Investigation, Formal analysis, Conceptualization. **Todor Stoyanov:** Writing – review & editing, Writing – original draft, Supervision, Formal analysis, Conceptualization. **Erik Schaffernicht:** Writing – review & editing, Writing – original draft, Supervision, Formal analysis, Conceptualization.

Declaration of competing interest

The authors affirm that there are no known competing interests or financial relationships that could be perceived as potential conflicts of interest.

Data availability

No data was used for the research described in the article.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used ChatGPT in order to improve readability and language. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Acknowledgments

This work was partially supported by the Swedish Foundation for Strategic Research (SSF) (project ID19-0053), the Industrial Graduate School Collaborative AI & Robotics (CoAIRob), funded by the Swedish Knowledge Foundation under Grant Dnr:20190128, and by the European Union's Horizon Europe Framework Programme under grant agreement No 101070596 (euROBIN).

References

- [1] M. Colledanchise, P. Ögren, Behavior Trees in Robotics and AI: an Introduction, 2018.
- [2] M. Iovino, E. Scutkins, J. Styrud, P. Ögren, C. Smith, A survey of behavior trees in robotics and AI, *Robot. Auton. Syst.* 154 (2022).
- [3] D. Isla, Handling complexity in the Halo 2 AI, 2005, https://www.gamasutra.com/view/feature/130663/gdc_2005_proceeding_handling.php?print=1.
- [4] M. Mateas, A. Stern, A behavior language for story-based believable agents, *IEEE Intell. Syst.* 17 (4) (2002) 39–47.
- [5] P. Ögren, Increasing modularity of UAV control systems using computer game behavior trees, in: *AIAA Guidance, Navigation, and Control Conference* 2012, 2012.
- [6] J.A. Bagnell, F. Cavalcanti, L. Cui, T. Galluzzo, M. Hebert, M. Kazemi, M. Klingensmith, J. Libby, T.Y. Liu, N. Pollard, M. Pivtoraiko, J.-S. Valois, R. Zhu, An integrated system for autonomous robotics manipulation, in: *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 2955–2962.
- [7] D. Cáceres Domínguez, M. Iannotta, J.A. Stork, E. Schaffernicht, T. Stoyanov, A stack-of-tasks approach combined with behavior trees: A new framework for robot control, *IEEE Robot. Autom. Lett.* 7 (4) (2022) 12110–12117.
- [8] Y. Cao, C.G. Lee, Behavior-tree embeddings for robot task-level knowledge, in: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2022*, pp. 12074–12080.
- [9] N. Li, H. Jiang, C. Li, Z. Wang, Towards adaptive behavior trees for robot task planning, in: *2022 China Automation Congress, CAC, 2022*, pp. 6720–6725.
- [10] T.G. Tadevos, L. Shamgah, A. Karimodini, Automatic decentralized behavior tree synthesis and execution for coordination of intelligent vehicles, *Knowl.-Based Syst.* (2023).
- [11] G. Heppner, N. Berg, D. Oberacker, N. Spielbauer, A. Roennau, R. Dillmann, Distributed behavior trees for heterogeneous robot teams, in: *2023 IEEE 19th International Conference on Automation Science and Engineering, CASE, 2023*, pp. 1–8.
- [12] J. Styrud, M. Iovino, M. Norrlöf, M. Björkman, C. Smith, Combining planning and learning of behavior trees for robotic assembly, in: *2022 International Conference on Robotics and Automation, ICRA, 2022*, pp. 11511–11517.
- [13] Z. Cai, M. Li, W. Huang, W. Yang, BT expansion: a sound and complete algorithm for behavior planning of intelligent robots with behavior trees, in: *Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, (7) 2021*, pp. 6058–6065.
- [14] P. Ögren, C.I. Sprague, Behavior trees in robot control systems, *Ann Rev Control, Robot, Autonomous Syst* 5 (2022) 81–107.
- [15] M. Colledanchise, L. Natale, Analysis and exploitation of synchronized parallel executions in behavior trees, in: *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2019*.
- [16] M. Iovino, J. Styrud, P. Falco, C. Smith, Learning behavior trees with genetic programming in unpredictable environments, in: *2021 IEEE International Conference on Robotics and Automation, ICRA, 2021*, pp. 4591–4597.
- [17] F. Roviola, B. Grossmann, V. Krüger, Extended behavior trees for quick definition of flexible robotic tasks, in: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2017*, pp. 6793–6800.
- [18] M. Colledanchise, A. Marzinotto, P. Ögren, Performance analysis of stochastic behavior trees, in: *2014 IEEE International Conference on Robotics and Automation, ICRA, 2014*, pp. 3265–3272.
- [19] M. Colledanchise, L. Natale, Handling concurrency in behavior trees, *IEEE Trans. Robot.* 38 (4) (2022) 2557–2576.
- [20] C. Paxton, A. Hundt, F. Jonathan, K. Guerin, G. Hager, Costar: Instructing collaborative robots with behavior trees and vision, 2017, pp. 564–571.
- [21] R. Wu, S. Kortik, C.H. Santos, Automated behavior tree error recovery framework for robotic systems, in: *2021 IEEE International Conference on Robotics and Automation, ICRA, 2021*, pp. 6898–6904.
- [22] S. Jeong, T. Ga, I. Jeong, J. Choi, Behavior tree driven multi-mobile robots via data distribution service (DDS), in: *2021 21st International Conference on Control, Automation and Systems, ICCAS, 2021*, pp. 1633–1638.
- [23] M. Colledanchise, G. Cicala, D.E. Domenichelli, L. Natale, A. Tacchella, Formalizing the Execution Context of Behavior Trees for Runtime Verification of Deliberative Policies.
- [24] R.d. Pereira, P.M. Engel, A framework for constrained and adaptive behavior-based agents, 2015, <https://arxiv.org/pdf/1506.02312>.
- [25] T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein, *Introduction to Algorithms*, third ed., 2009.
- [26] C. Paduraru, M. Paduraru, Automatic difficulty management and testing in games using a framework based on behavior trees and genetic algorithms, in: *2019 24th International Conference on Engineering of Complex Computer Systems, ICECCS, 2019*, pp. 170–179.
- [27] A. Wathieu, T.R. Groechel, H.J. Lee, C. Kuo, M.J. Mataric, RE:BT-Espresso: Improving interpretability and expressivity of behavior trees learned from robot demonstrations, in: *2022 International Conference on Robotics and Automation, ICRA, 2022*, pp. 11518–11524.
- [28] G. Robertson, I. Watson, Building behavior trees from observations in real-time strategy games, in: *2015 International Symposium on Innovations in Intelligent SysTems and Applications, INISTA, 2015*, pp. 1–7.
- [29] A. Sanfeliu, K.-S. Fu, A distance measure between attributed relational graphs for pattern recognition, *IEEE Trans. Syst. Man Cybern. SMC-13* (1983) 353–362.
- [30] M. Iovino, J. Förster, P. Falco, J.J. Chung, R. Siegwart, C. Smith, On the programming effort required to generate behavior trees and finite state machines for robotic applications, in: *2023 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2023*, pp. 5807–5813.
- [31] S. Gugliermo, E. Schaffernicht, C. Koniaris, F. Pecora, Learning behavior trees from planning experts using decision tree and logic factorization, *IEEE Robot. Autom. Lett.* (2023) 3534–3541.
- [32] A.R. Nogueira, A. Pugnana, S. Ruggieri, D. Pedreschi, J. Gama, Methods and tools for causal discovery and causal inference, *Wiley Interdiscip. Rev.: Data Min. Knowl. Discov.* (2022).
- [33] O. Biggar, M. Zamani, I. Shames, A principled analysis of behavior trees and their generalisations, 2021, [arXiv: https://arxiv.org/abs/2008.11906](https://arxiv.org/abs/2008.11906).
- [34] X. Neufeld, S. Mostaghim, S. Brand, A hybrid approach to planning and execution in dynamic environments through hierarchical task networks and behavior trees, in: *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, 2018*, pp. 201–207.
- [35] M. Colledanchise, P. Ögren, How behavior trees modularize robustness and safety in hybrid systems, in: *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2014*, pp. 1482–1488.
- [36] IEEE standard glossary of software engineering terminology, in: *IEEE Std 610.12-1990*, 1990, pp. 1–84.
- [37] Q. Huang, X. Ma, K. Liu, X. Ma, W. Pang, Autonomous reconnaissance action of swarm unmanned system driven by behavior tree, in: *2022 IEEE International Conference on Unmanned Systems, ICUS, 2022*, pp. 1540–1544.
- [38] Z. Manna, The correctness of programs, *J. Comput. System Sci.* (1969) 119–127.
- [39] Z. Gu, N. Boyd, Y. Zhao, Reactive locomotion decision-making and robust motion planning for real-time perturbation recovery, in: *2022 International Conference on Robotics and Automation, ICRA, 2022*, pp. 1896–1902.
- [40] J. Xu, Y. Lin, H. Zhou, H. Min, Generating manipulation sequences using reinforcement learning and behavior trees for peg-in-hole task, in: *2022 IEEE International Conference on Systems, Man, and Cybernetics, SMC, 2022*, pp. 2715–2720.
- [41] W.J. Murdoch, C. Singh, K. Kumbier, R. Abbasi-Asl, B. Yu, Definitions, methods, and applications in interpretable machine learning, in: *Proceedings of the National Academy of Sciences, 2019*, pp. 22071–22080.
- [42] K. French, S. Wu, T. Pan, Z. Zhou, O.C. Jenkins, Learning behavior trees from demonstration, in: *2019 International Conference on Robotics and Automation, ICRA, 2019*, pp. 7791–7797.
- [43] S. Shu, S. Preum, H.M. Pitchford, R.D. Williams, J. Stankovic, H. Alemzadeh, A behavior tree cognitive assistant system for emergency medical services, in: *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2019*, pp. 6188–6195.
- [44] S. Jones, M. Studley, S. Hauert, A. Winfield, Evolving behaviour trees for swarm robotics, in: *Distributed Autonomous Robotic Systems: The 13th International Symposium*, Springer, 2018, pp. 487–501.
- [45] C. Paxton, F. Jonathan, A. Hundt, B. Mutlu, G.D. Hager, User experience of the CoSTAR system for instruction of collaborative robots, 2017, <https://arxiv.org/pdf/1703.07890>.
- [46] H.C. Siu, K. Leahy, M. Mann, STL: Surprisingly tricky logic (for system validation), in: *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2023*, pp. 8613–8620.
- [47] P. Tulathum, B. Usawalerkamol, G.A.G. Ricardez, J. Takamatsu, T. Ogasawara, K. Matsumoto, Human-robot interaction system for non-expert users in convenience stores using behavior trees, in: *2022 IEEE/SICE International Symposium on System Integration, SII, 2022*, pp. 1072–1077.
- [48] K.Y.W. Scheper, S. Tijmons, C.C. de Visser, G.C.H.E. de Croon, Behaviour trees for evolutionary robotics, 2014, [arXiv: https://arxiv.org/pdf/1411.7267](https://arxiv.org/pdf/1411.7267).
- [49] M. Colledanchise, R. Parasuraman, P. Ögren, Learning of behavior trees for autonomous agents, *IEEE Trans. Games* 11 (2) (2019) 183–189.
- [50] B. Hannaford, D. Hu, D. Zhang, Y. Li, Simulation results on selector adaptation in behavior trees, 2016, <https://arxiv.org/pdf/1606.09219>.
- [51] Y. Ma, C. Deng, J. Zhang, Y. Wu, H. Jin, Y. Wang, Resource scheduling in behavior trees, in: *2022 IEEE International Conference on Robotics and Biomimetics, ROBIO, 2022*, pp. 2287–2292.

- [52] A. Rastegarpanah, H.C. Gonzalez, R. Stolkin, Semi-autonomous behaviour tree-based framework for sorting electric vehicle batteries components, *Robotics* (2021).
- [53] F. Fusaro, E. Lamon, E.D. Momi, A. Ajoudani, A human-aware method to plan complex cooperative and autonomous tasks using behavior trees, in: 2020 IEEE-RAS 20th International Conference on Humanoid Robots, *Humanoids*, 2021, pp. 522–529.
- [54] M. Stuede, T. Lerche, M.A. Petersen, S. Spindeldreier, Behavior-tree-based person search for symbiotic autonomous mobile robot tasks, in: 2021 IEEE International Conference on Robotics and Automation, *ICRA*, 2021, pp. 2414–2420.
- [55] H. Soebhakti, M. Fauzi, T. Ata, W. Kurniawan, Coach application for soccer robot, in: Proceedings of the 6th International Conference on Applied Engineering, *ICAE* 2023.
- [56] G. Kokotinis, G. Michalos, Z. Arkouli, S. Makris, A behavior trees-based architecture towards operation planning in hybrid manufacturing, *Taylor & Francis*, 2024, pp. 324–349.
- [57] E. Safronov, M. Vilzmann, D. Tsetserukou, K. Kondak, Asynchronous behavior trees with memory aimed at aerial vehicles with redundancy in flight controller, in: 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems, *IROS*, IEEE, 2019, pp. 3113–3118.
- [58] F. Martín Rico, M. Morelli, H. Espinoza, F.J. Rodríguez-Lera, V. Matellán Olivera, Optimized execution of PDDL plans using behavior trees, in: Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems, *AAMAS '21*, International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 2021, pp. 1596–1598.
- [59] Y. Wu, J. Li, H. Dai, X. Yi, Y. Wang, X. Yang, micROS.BT: An event-driven behavior tree framework for swarm robots, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems, *IROS*, 2021, pp. 9146–9153.
- [60] S. Yi, S. Worrall, E. Nebot, A persistent and context-aware behavior tree framework for multi sensor localization in autonomous driving, 2021, <https://arxiv.org/pdf/2103.14261>.
- [61] S. Yang, X. Mao, S. Wang, Y. Bai, Extending behavior trees for representing and planning robot adjoint actions in partially observable environments, *J. Intell. Robot. Syst.* 102 (2021).
- [62] Q. Yang, Z. Luo, W. Song, R. Parasuraman, Self-reactive planning of multi-robots with dynamic task assignments, in: 2019 International Symposium on Multi-Robot and Multi-Agent Systems, *MRS*, IEEE, 2019, pp. 89–91.
- [63] K. Fozilov, Y. Hasegawa, K. Sekiyama, Towards self-autonomy evaluation using behavior trees, in: 2021 IEEE International Conference on Systems, Man, and Cybernetics, *SMC*, 2021, pp. 988–993.
- [64] P. de la Cruz, J. Piater, M. Saveriano, Reconfigurable behavior trees: Towards an executive framework meeting high-level decision making and control layer features, in: 2020 IEEE International Conference on Systems, Man, and Cybernetics, *SMC*, 2020, pp. 1915–1922.
- [65] C. Paxton, N. Ratliff, C. Eppner, D. Fox, Representing robot task plans as robust logical-dynamical systems, in: 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems, *IROS*, IEEE, 2019, pp. 5588–5595.
- [66] L. Castano, H. Xu, Safe decision making for risk mitigation of UAS, in: 2019 International Conference on Unmanned Aircraft Systems, *ICUAS*, 2019, pp. 1326–1335.
- [67] M. Mayr, K. Chatzilygeroudis, F. Ahmad, L. Nardi, V. Krueger, Learning of parameters in behavior trees for movement skills, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems, *IROS*, 2021, pp. 7572–7579.
- [68] J. Kuckling, A. Ligot, D. Bozhinoski, M. Birattari, Behavior trees as a control architecture in the automatic modular design of robot swarms, in: M. Dorigo, M. Birattari, C. Blum, A.L. Christensen, A. Reina, V. Trianni (Eds.), *Swarm Intelligence*, Springer International Publishing, Cham, 2018, pp. 30–43.
- [69] C. Gao, Y. Zhai, B. Wang, B.M. Chen, Synthesis and online re-planning framework for time-constrained behavior tree, in: 2021 IEEE International Conference on Robotics and Biomimetics, *ROBIO*, 2021, pp. 1896–1901.
- [70] H. Zhou, H. Min, Y. Lin, An autonomous task algorithm based on behavior trees for robot, in: 2019 2nd China Symposium on Cognitive Computing and Hybrid Intelligence, *CCHI*, IEEE, 2019, pp. 64–70.
- [71] F. Rovida, D. Wuthier, B. Grossmann, M. Fumagalli, V. Krüger, Motion generators combined with behavior trees: A novel approach to skill modelling, in: 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, *IROS*, 2018, pp. 5964–5971.

- [72] T.G. Tadewos, A.A. Redwan Newaz, A. Karimoddini, Specification-guided behavior tree synthesis and execution for coordination of autonomous systems, *Expert Systems with Applications* 201 (2022).



Simona Gugliermo received her M.Sc. in Systems, Control, and Robotics from the Royal Institute of Technology (KTH) in Stockholm in 2019. She is currently employed at Scania CV AB (SE), while pursuing a Ph.D. in Computer Science at the Centre for Applied Autonomous Sensor Systems (AASS) at Örebro University. Her primary research revolves around AI planning for fleets of autonomous vehicles, with a focus on integrating learning techniques for synthesizing planning domains.



David Cáceres Domínguez received a master's degree in Science and Technology Studies from the Technical University of Munich (TUM) in 2020. He is currently pursuing a Ph.D. in Computer Science at the Centre for Applied Autonomous Sensor Systems (AASS) at Örebro University in collaboration with Suzuki Garphyttan. His research interests include control and planning for collaborative mobile manipulation and interactive robot learning.



Marco Iannotta received his M.Sc. in Computer Science from the University of Bari (IT) in 2020, with a specialization in Knowledge Engineering and Machine Intelligence. He is currently pursuing a Ph.D. in Computer Science at the Centre for Applied Autonomous Systems (AASS) at Örebro University, in collaboration with Suzuki Garphyttan. He is mainly interested in control and planning in dynamic industrial environments, with a focus on employing learning techniques to enhance the adaptivity of robot policies.



Todor Stoyanov is Associate Prof. of computer science at Örebro University, Sweden. His research interests span mobile manipulation, 3D perception, robot autonomy and robot learning.



Erik Schaffernicht is a senior lecturer in AI and robotics at Örebro University in Sweden. He received his Diploma degree in computer science (2006) and his Doctoral degree in neuroinformatics (2012) both from Ilmenau University of Technology, Germany. He is co-author of more than 70 peer reviewed publications and a patent. His research interests range from mobile robot olfaction to feature selection methods and reinforcement learning.